

PROJECT REPORT: VEHICLE SERVICE MANAGEMENT SYSTEM (VSMS)

1. Cover Page

- **Project Title:** Layered Vehicle Service & Garage Operations Management System (VSMS)
- **Course Name:** Object-Oriented Programming / Advanced Java Project Evaluation
- **Student Name:** Jasmine Chauhan
- **Registration Number:** 25BAI11279
- **Faculty Name:** Sandeep Monga
- **Institution:** Vellore Institute of Technology (VIT) / VITyarthi

2. Introduction

The automotive repair and garage management industry relies heavily on tracking vehicle diagnostics, spare component consumption, labor hours, and multi-tier tax accounting. The **Vehicle Service Management System (VSMS)** is a standalone, 5-tier layered desktop software engineered with **Java 17 LTS, JavaFX, SQLite JDBC, and JUnit 5**.

ZIP+ 1

The system automates garage reception, fleet maintenance, job scheduling, line-item parts rollups, and invoice settlements without external application servers. Built around parameterized database transactions and strict Object-Oriented Programming (OOP) paradigms, VSMS delivers interactive analytics through JavaFX data charts (PieChart, BarChart) styled with a clean CSS3 theme (style.css).

ZIP+ 1

3. Problem Statement

Independent auto repair workshops and multi-brand service centers routinely suffer from operational bottlenecks:

1. **Fragmented Service Records:** Lack of unified tracking results in lost vehicle service histories and duplicate registration entries.

ZIP

2. **Inaccurate Billing Calculations:** Manual calculation of consumable parts, technician labor, and statutory sales tax (18% GST) causes financial discrepancies and client disputes.

ZIP

3. **Workflow Skipping:** Invoicing uncompleted repair work or adding parts to settled bills creates inventory and audit inconsistencies.

ZIP

4. **Poor Management Visibility:** Garages lack instant metrics regarding active jobs, completed jobs, gross revenue, and pending receivables.

ZIP

VSMS resolves these vulnerabilities with automated state-machine validations, atomic SQLite transactions, parameterized queries, and real-time dashboard visual aggregations.

ZIP

4. Functional Requirements

The system provides five dedicated functional modules:

ZIP

Module 1: Staff Authentication & Access Control

- **Credential Verification:** Multi-role staff authentication (System Administrator, Service Advisor, Workshop Technician) utilizing SHA-256 cryptographic password hashing (`DatabaseConfig.hashPassword()`).

ZIP

- **Session Tracking:** Dynamic UI rendering reflecting active staff credentials and roles upon login (`AuthService`).

ZIP

Module 2: Customer Relationship & Vehicle Fleet Tracking

- **Customer Management:** Full CRUD capabilities with real-time live search across customer names, contact phone numbers, and emails.

ZIP

- **Fleet Registry:** Unique vehicle identification enforcement preventing duplicate registration plates (e.g., KA01AB1234), categorized by vehicle class (Sedan, SUV, Hatchback, Motorcycle) and validated manufacturing years.

ZIP

- **Multi-Vehicle Association:** Relational one-to-many cascading binds enabling single customers to own and service multiple vehicles.

ZIP

Module 3: Service Request & Lifecycle Progression

- **Job Card Issuance:** Creating structured service tickets detailing reported vehicle faults, intake dates, and estimated technician labor fees.

ZIP

- **Status State Machine:** Strictly managed lifecycle progression:

Pending→In Progress→Completed (or Cancelled)

ZIP

- **Consumables & Parts Ledger:** Dynamic line-item spare parts tracking with automatic cost rollups:

ZIP

Item Total=Quantity×Unit Price

Module 4: Automated Billing & Invoicing Engine

- **Formula Enforcement:** Strict, automated calculation ensuring zero calculation error:

ZIP

Subtotal=Parts Cost+Labour Cost

GST (18%)=Subtotal×0.18

Total Invoice Amount=Subtotal+GST

- **Workflow Guard:** Precondition ensuring only **Completed** jobs can trigger invoice generation.

ZIP

- **Settlement Tracking:** Single-click payment status toggle (Unpaid → Paid).

ZIP

Module 5: Executive Dashboard & Operational Analytics

- **KPI Metrics:** Real-time calculation of registered clients, vehicles, open repair orders, total revenue collected, and pending payments.

ZIP

- **JavaFX Visualizations:** Dynamic PieChart representing job status distributions and BarChart comparing collected versus outstanding revenue.

ZIP

5. Non-Functional Requirements

1. **Security & SQL Injection Prevention:** Complete isolation from SQL injection vulnerabilities using parameterized JDBC PreparedStatement interfaces across all DAOs (CustomerDAO, VehicleDAO, ServiceDAO, BillDAO, UserDAO).

ZIP

2. **Data Integrity & Relational Constraints:** Native enforcement of foreign key referential integrity (PRAGMA foreign_keys = ON;), unique constraints, and check constraints (payment_status IN ('Unpaid', 'Paid')).

ZIP

3. **Usability & Modern Desktop Interface:** JavaFX desktop UI featuring responsive layouts (BorderPane, StackPane, VBox), custom CSS styling, and clear operational badge markers (badge-pending, badge-completed).

ZIP

4. **Maintainability & Architectural Modularity:** Clean 5-tier separation (model, dao, service, controller, util) ensuring independent component testability.

ZIP

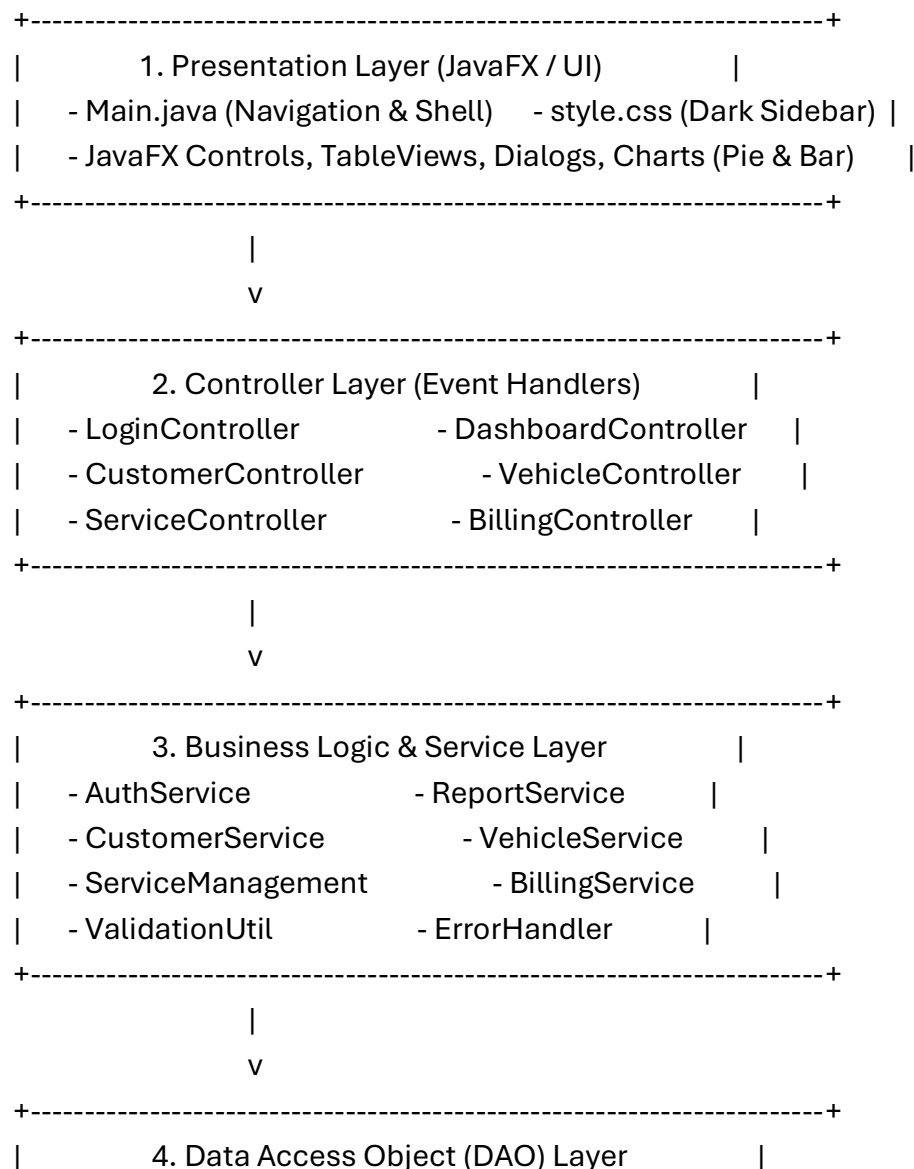
5. **Reliability & Error Handling:** Graceful exception interception (ErrorHandler.java) translating raw SQL exceptions into user-friendly diagnostic dialogs.

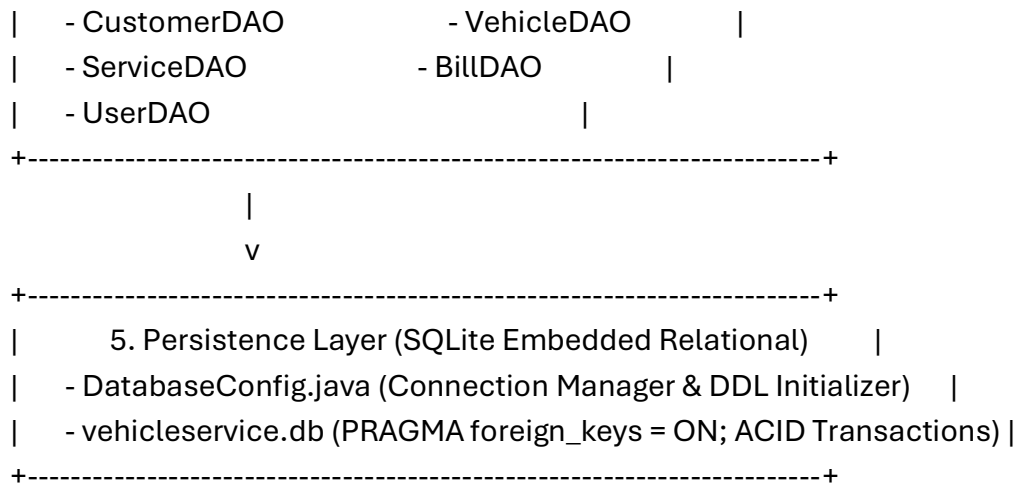
ZIP

6. System Architecture

VSMS utilizes a strict **5-Tier Layered Architecture**:

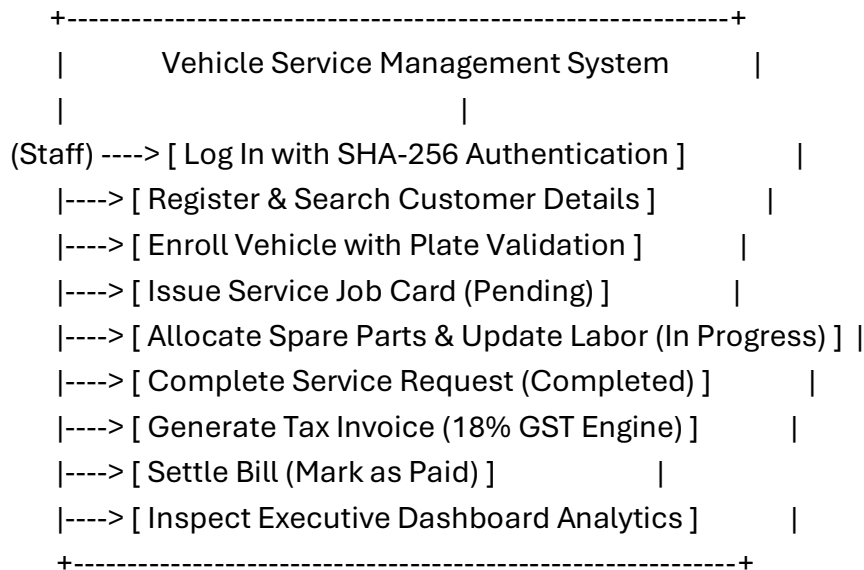
ZIP



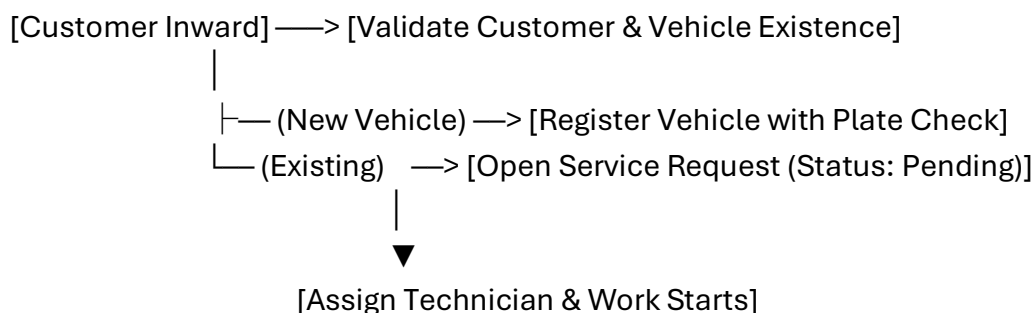


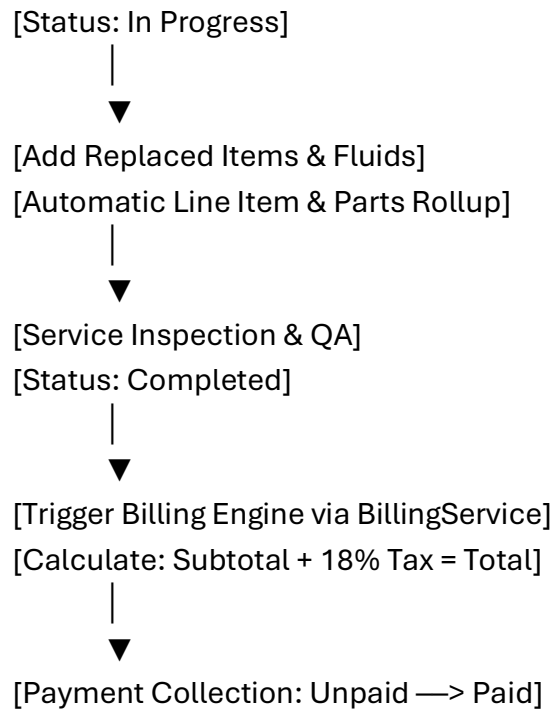
7. Design Diagrams

7.1 Use Case Diagram

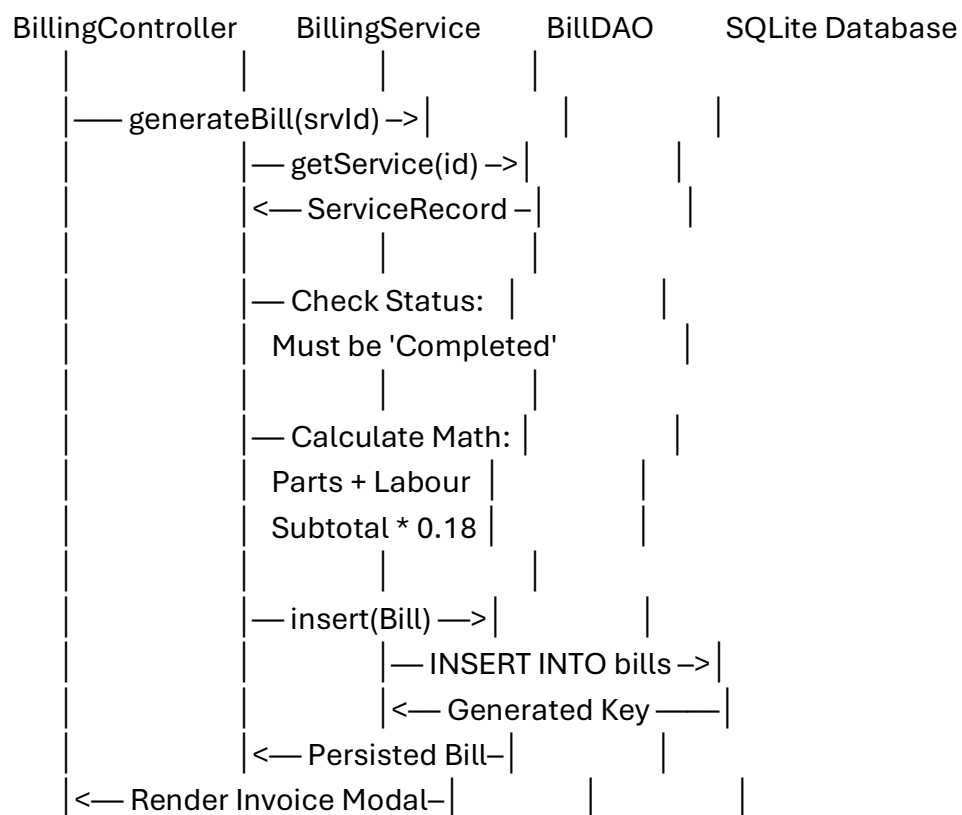


7.2 Process Flow / Workflow Diagram (Workshop Lifecycle)





7.3 Sequence Diagram (Invoice Generation)



7.4 Class / Component Diagram

- **Core Models (com.vehicleservice.model):**

- Customer: id, name, phone, email, address.

ZIP

- Vehicle: id, customerId, registrationNumber, brand, model, vehicleType, manufacturingYear.

ZIP

- ServiceRecord: id, vehicleId, serviceType, description, serviceDate, status, labourCost, partsCost.

ZIP

- ServiceItem: id, serviceId, itemName, quantity, unitPrice, totalPrice.

ZIP

- Bill: id, serviceId, subtotal, tax, totalAmount, paymentStatus, billDate.

ZIP

- User: id, username, passwordHash, fullName, role.

ZIP

- **Service Orchestrators (com.vehicleservice.service):**

- BillingService: Executes arithmetic formulas and tax logic.

ZIP

- ServiceManagement: Manages lifecycle states and automatic rollups.

ZIP

- ReportService: Computes aggregates into DashboardSummary transfer objects.

ZIP

7.5 ER Diagram (Relational Database Design)

- **users** (id [PK], username [UNIQUE], password_hash, full_name, role, created_at)

ZIP

- **customers** (id [PK], name, phone [INDEXED], email, address, created_at)

ZIP

- **vehicles** (id [PK], customer_id [FK → customers.id ON DELETE CASCADE], registration_number [UNIQUE, INDEXED], brand, model, vehicle_type, manufacturing_year)

ZIP

- **service_records** (id [PK], vehicle_id [FK → vehicles.id ON DELETE CASCADE], service_type, description, service_date, status [CHECK: 'Pending', 'In Progress', 'Completed', 'Cancelled'], labour_cost, parts_cost)

ZIP

- **service_items** (id [PK], service_id [FK → service_records.id ON DELETE CASCADE], item_name, quantity, unit_price, total_price)

ZIP

- **bills** (id [PK], service_id [UNIQUE FK → service_records.id ON DELETE CASCADE], subtotal, tax, total_amount, payment_status [CHECK: 'Unpaid', 'Paid'], bill_date)

ZIP

8. Design Decisions & Rationale

1. Pure OpenJFX & Native FXML-Free Modular Controllers:

- a. *Rationale:* Constructing dynamic, reusable controls programmatically in Java controllers provides compile-time safety, refactoring agility, and eliminates FXML binding desynchronization.

ZIP

2. SQLite Embedded Database with Foreign Key Enforcement:

- a. *Rationale:* Embedded SQLite eliminates the need for evaluators to install, configure, or start standalone MySQL/PostgreSQL server daemons while delivering complete ACID compliance and relational integrity via PRAGMA foreign_keys = ON;.

ZIP

3. Automated Rollups via Data Layer Callbacks:

- a. *Rationale:* When spare parts are added or deleted in ServiceDAO, the database executes recalculatePartsCost() automatically using SQL aggregate functions (COALESCE(SUM(total_price), 0.0)), guaranteeing parts costs remain synchronized with line items.

ZIP

9. Implementation Details

- **Technology Stack:** OpenJDK 17 LTS, JavaFX 17.0.10, SQLite JDBC 3.45.1.0, JUnit 5 Jupiter 5.10.2, SLF4J, Apache Maven 3.9+.

ZIP

- **Input Validation Engine (ValidationUtil.java):** Regular expressions sanitize email addresses, contact numbers (10–13 digits), vehicle plate formats, and restrict manufacturing years to valid historical boundaries ($1900 \leq \text{year} \leq \text{current year} + 1$).

ZIP

- **Self-Bootstrapping Architecture (Main.java):** On startup, DatabaseConfig.initializeDatabase() parses and executes schema.sql, provisions performance indexes, and seeds realistic test entities and default admin credentials (admin / admin123) if the database is unpopulated.

ZIP

10. Screenshots / Results

(Note: Capture and paste the following 4 screenshots here before PDF generation)

1. **Staff Login Portal:** Modern authentication dialog with secure masked input fields.

ZIP

2. **Executive Dashboard:** Live KPI cards alongside the JavaFX PieChart (Job Status Breakdown) and BarChart(Collected vs Pending Revenue).

ZIP

3. **Service Management & Line-Item Parts Ledger:** Detailed job view showing spare parts allocation and status transition controls.

ZIP

4. **Tax Billing Modal & Test Suite Execution:** Itemized tax invoice display and console summary showing **31 passing tests** from TestSuiteRunner.

ZIP

11. Testing Approach

The application features a test suite containing **31 automated JUnit 5 tests** across unit, constraint, and end-to-end integration tiers:

ZIP

Test Class	Scope	What Was Verified
CustomerDAOTest	DAO Unit Test	Customer insertion, phone search, field updates, cascading safety.
		ZIP
VehicleDAOTest	DAO Unit Test	Duplicate plate rejection, customer fleet lookups, year constraints.
		ZIP
ServiceDAOTest	DAO Unit Test	Status state machine transitions, parts cost sum rollups.
		ZIP
BillingServiceTest	Arithmetic Unit	Sum of parts, 18% GST computation, negative cost rejection.
		ZIP

ValidationUtilTest	Utility Unit	Phone/email regex validations and boundary year assertions. ZIP
DatabaseErrorHandlingTest	DB Integrity	Foreign key constraint failures and invalid status check assertions. ZIP
IntegrationWorkflowTest	End-to- End	Complete business lifecycle: Customer → Vehicle → Job → Parts → Invoice → Paid. ZIP

12. Challenges Faced

1. Cascading Cost Synchronization on Parts Modification:

- Challenge:* Modifying, adding, or removing line items frequently led to out-of-sync subtotal values in the parent service record.
- Resolution:* Engineered recalculatePartsCost() within ServiceDAO using SQL aggregate queries wrapped in atomic transactions whenever an item is added or removed.

ZIP

2. Preventing Premature Billing Inconsistencies:

- Challenge:* Staff could potentially generate invoices while mechanics were still adding parts to an active job.
- Resolution:* Built precondition assertions into BillingService.generateBill() that reject any service ticket not explicitly marked as Completed.

ZIP

13. Learnings & Key Takeaways

- Designed and implemented enterprise-level **Layered Java Architecture (5-Tier)** with clean decoupling between presentation, controllers, business services, and database persistence.

ZIP

- Mastered **Parameterized JDBC Statements** to prevent SQL injection vulnerabilities and handle relational database constraints cleanly.

ZIP

- Leveraged JavaFX layout containers and CSS styling to build a modern, responsive desktop interface with data visualizations.

ZIP

14. Future Enhancements

- **Automated Customer Telecommunications:** WhatsApp and SMS notification triggers via Twilio API when repair tickets transition to "Ready for Delivery".

ZIP

- **Digital Payment Gateway Integration:** Dynamic UPI QR-code rendering on invoice inspection for instant digital payments.

ZIP

- **Low-Stock Inventory Warnings:** Component stock deduction with reorder triggers when parts inventory drops below minimum thresholds.

ZIP

- **PDF Invoice Generation:** Direct export of tax invoices to customer-ready PDF files using Apache PDFBox or OpenPDF.

ZIP

15. References

1. Oracle Corporation. *Java SE 17 Documentation & Language Specification*, 2026.

ZIP

2. OpenJFX Community. *JavaFX 17 Developer Documentation & Component Reference*, 2026.

ZIP

3. SQLite Consortium. *SQLite 3 SQL Language Rules and Foreign Key Support*, 2026.

ZIP

4. JUnit Team. *JUnit 5 User Guide & Testing Framework Documentation*, 2026.

ZIP

5. VITyarthi Project Guidelines & Rubric, *Build Your Own Project Instructions*, 2026.

PDF+ 1